

PyQt5 图标配置 & 打包命令文档

桌面宠物 · 团子陪伴 v7.0.0

版本: V1.0

日期: 2026 年 8 月

文档类型: 技术配置文档

目录

一、概述	4
目标读者	4
二、应用图标素材准备	4
2.1 图标尺寸规范	4
2.2 PNG 源文件要求	5
三、macOS icns 图标制作	5
3.1 iconutil 命令行制作（系统自带，无依赖）	5
步骤 1：准备 .iconset 目录	6
步骤 2：使用 iconutil 转换	6
步骤 3：预览验证	7
3.2 手动制作 icns（进阶，需要 Pillow）	7
3.3 在线工具转换（备用方案）	8
四、Windows ICO 图标制作	8
4.1 Pillow 生成 ICO（推荐）	8
4.2 ImageMagick 转换	9
五、PyQt5 代码中设置图标	9
5.1 应用窗口图标	9
单独窗口设置图标（非全局继承）	10
5.2 对话框/弹窗图标	11
5.3 菜单项图标与 Emoji	12
使用 QIcon + PNG 的品牌化菜单	13
5.4 前端网页图标配置	14
favicon 浏览器标签页图标	14
各 Vue 组件中引用	14
六、Info.plist 图标配置	15
6.1 CFBundleIconFile 配置	15
6.2 完整配置示例 + icns 复制到 .app	16
七、PyInstaller 打包命令详解	17
7.1 基础打包命令	17

main.py 适配 PyInstaller 的资源路径写法	17
7.2 打包参数速查表	18
7.3 Spec 文件配置	19
八、py2app 打包 (macOS 原生)	22
8.1 setup.py 配置	22
8.2 py2app 打包命令	25
九、build_app.sh 脚本打包 (本项目推荐)	25
9.1 一键打包命令	25
9.2 脚本参数说明	26
9.3 与 PyInstaller/py2app 对比	27
十、代码签名与公证流程	27
10.1 Ad-hoc 签名 (无开发者账号)	27
10.2 Developer ID 签名 (Apple 开发者账号 \$99/年)	28
10.3 Notarization 公证 (提交 Apple 扫描)	28
10.4 Stapler 装订公证结果	29
十一、DMG 安装包制作	30
11.1 hdiutil 命令制作 (系统自带)	30
11.2 create-dmg 工具美化 (推荐)	30
十二、常见问题排查	31
12.1 图标不显示	31
12.2 打包启动失败	32
错误: No module named config / pet_window	32
错误: Could not find the Qt platform plugin "cocoa"	32
双击 .app 一闪而过 (命令行运行有报错)	32
12.3 签名与公证错误	32
十三、附录: 完整打包命令速查	33

一、概述

本文档详细介绍《团子陪伴》桌面宠物应用的 PyQt5 图标配置方案与打包命令全流程。涵盖从 icns/ICO 图标素材制作、PyQt5 代码中图标设置方法、Info.plist 配置，到 PyInstaller、py2app、build_app.sh 三种打包方式的命令详解，以及代码签名、公证、DMG 制作的完整指南。

目标读者

- 桌面端开发人员：需要配置应用图标与打包发布
- UI/UX 设计师：了解图标导出规范与格式要求
- 运维与测试人员：执行打包发布流程与问题排查

二、应用图标素材准备

2.1 图标尺寸规范

macOS .app 图标 (.icns) 必须包含以下全部 10 种尺寸：

尺寸 (px)	文件名	DPI 用途	说明	建议场景
16 × 16	icon_16x16.png	72dpi @1x	侧边栏/小列表视图	极简剪影可识别即可
32 × 32	icon_16x16@2x.png	144dpi @2x	16 寸 Retina 对应 32px	保留少量细节
32 × 32	icon_32x32.png	72dpi @1x	中等列表视图	
64 × 64	icon_32x32@2x.png	144dpi @2x	32 寸 Retina 对应 64px	
128 × 128	icon_128x128.png	72dpi @1x	网格视图 (中等)	开始呈现主体细节
256 × 256	icon_128x128@2x.png	144dpi @2x	128 寸 Retina 对应 256px	
256 × 256	icon_256x256.png	72dpi @1x	大网格视图	丰富的细节与质感
512 × 512	icon_256x256@2x.png	144dpi @2x	256 寸 Retina 对应 512px	

512 × 512	icon_512x512.png	72dpi @1x	Cover Flow/预览大图	几乎所有细节可见
1024 × 1024	icon_512x512@2x.png	144dpi @2x	5K Retina/商店展示图	最高精度，原图导出

2.2 PNG 源文件要求

格式： PNG-24 或 PNG-32（带 Alpha 透明通道）

色彩空间： sRGB（Display P3 广色域需额外配置，不推荐）

背景： 必须透明（猫咪头像无方形背景），绝不能用 JPG 或不透明 PNG

圆角： macOS 自动裁切圆角，图标不需要手动画圆角；但安全区域建议保留 10% 边距

输出： 10 张 PNG 分别导出，不要使用单张 1024 让系统缩放（会模糊）

推荐目录结构

```
tuanzi_icons/  
├── icon.iconset/      # 注意：必须是 .iconset 后缀的目录  
│   ├── icon_16x16.png  
│   ├── icon_16x16@2x.png  
│   ├── icon_32x32.png  
│   ├── icon_32x32@2x.png  
│   ├── icon_128x128.png  
│   ├── icon_128x128@2x.png  
│   ├── icon_256x256.png  
│   ├── icon_256x256@2x.png  
│   ├── icon_512x512.png  
│   └── icon_512x512@2x.png
```

【注意】 文件名大小写必须严格一致！iconutil 对文件名敏感，错一个字母就会缺尺寸或生成失败。

三、macOS icns 图标制作

3.1 iconutil 命令行制作（系统自带，无依赖）

macOS 自带 iconutil 工具，是制作 .icns 的标准方式：

步骤 1: 准备 .iconset 目录

```
# 创建 .iconset 目录

mkdir -p tuanzi_icons/icon.iconset

cd tuanzi_icons

# 假设设计师提供了一张 1024x1024 的 master.png
# 用 sips (系统自带) 批量生成全部 10 个尺寸

sips -z 16 16 master.png --out icon.iconset/icon_16x16.png
sips -z 32 32 master.png --out icon.iconset/icon_16x16@2x.png
sips -z 32 32 master.png --out icon.iconset/icon_32x32.png
sips -z 64 64 master.png --out icon.iconset/icon_32x32@2x.png
sips -z 128 128 master.png --out icon.iconset/icon_128x128.png
sips -z 256 256 master.png --out icon.iconset/icon_128x128@2x.png
sips -z 256 256 master.png --out icon.iconset/icon_256x256.png
sips -z 512 512 master.png --out icon.iconset/icon_256x256@2x.png
sips -z 512 512 master.png --out icon.iconset/icon_512x512.png
# 最后一张就用原图 (1024x1024)

cp master.png icon.iconset/icon_512x512@2x.png

# 验证 10 个尺寸都有

ls -la icon.iconset/

# 预期输出: 10 个 png 文件
```

步骤 2: 使用 iconutil 转换

```
# 在 icon.iconset 的上级目录执行

iconutil -c icns icon.iconset -o tuanzi.icns

# 验证生成

ls -lh tuanzi.icns

# 预期大小: 约 300KB ~ 1MB
```

步骤 3: 预览验证

方法 1: Finder 中按空格预览

```
open tuanzi.icns
```

方法 2: 预览全部尺寸

```
sips -g all tuanzi.icns
```

方法 3: 临时替换某个 .app 图标测试

右键 桌面宠物.app → 显示简介 → 将 tuanzi.icns 拖到左上角图标处

3.2 手工制作 icns (进阶, 需要 Pillow)

如果需要更精细的缩放控制 (使用 Lanczos 而非 sips 默认算法) :

```
#!/usr/bin/env python3
"""从 master.png 生成全部尺寸并打包 icns"""
from PIL import Image
import os, subprocess

MASTER = "master.png"
OUT_DIR = "icon.iconset"
os.makedirs(OUT_DIR, exist_ok=True)

sizes = [
    ("icon_16x16.png",      16),
    ("icon_16x16@2x.png",   32),
    ("icon_32x32.png",      32),
    ("icon_32x32@2x.png",   64),
    ("icon_128x128.png",    128),
    ("icon_128x128@2x.png", 256),
    ("icon_256x256.png",    256),
    ("icon_256x256@2x.png", 512),
    ("icon_512x512.png",    512),
```

```

("icon_512x512@2x.png", 1024),
]

img = Image.open(MASTER).convert("RGBA")
for fname, size in sizes:
    # LANCZOS = 高质量 Lanczos 缩采样
    resized = img.resize((size, size), Image.LANCZOS)
    resized.save(os.path.join(OUT_DIR, fname), "PNG")
    print(f" {fname} {size}x{size}")

# 调用 iconutil 打包
subprocess.run(["iconutil", "-c", "icns", OUT_DIR, "-o", "tuanzi.icns"])
print("
    已生成 tuanzi.icns")

```

3.3 在线工具转换（备用方案）

- <https://cloudconvert.com/png-to-icns> —— 上传 master.png, 直接导出 .icns
- <https://iconverticons.com/online/> —— 支持 PNG → ICNS/ICO 批量
- <https://www.img2icns.com/> —— 拖拽上传, 一键下载

【提示】 在线工具的缩放算法不如本地 sips/Pillow 精细, 且上传原图可能泄露版权, 仅作临时测试用。

四、Windows ICO 图标制作

4.1 Pillow 生成 ICO（推荐）

```

from PIL import Image

# Windows 标准 ICO 必须包含以下尺寸（从大到小）
sizes = [(256, 256), (128, 128), (64, 64), (48, 48),
         (32, 32), (24, 24), (20, 20), (16, 16)]

img = Image.open("master.png").convert("RGBA")
images = []

```



```

for size in sizes:
    resized = img.resize(size, Image.LANCZOS)
    images.append(resized)

# 保存为多尺寸 ICO
images[0].save(
    "tuanzi.ico",
    format="ICO",
    sizes=sizes,
    append_images=images[1:],
)
print("    tuanzi.ico 已生成")

```

4.2 ImageMagick 转换

```

# 安装 ImageMagick (brew install imagemagick)
convert master.png -define icon:auto-resize="256,128,64,48,32,24,20,16" tuanzi.ico

# 验证包含的尺寸
identify tuanzi.ico

```

五、PyQt5 代码中设置图标

5.1 应用窗口图标

在 main.py 中设置 QApplication 的主图标，所有窗口默认继承此图标：

```

"""main.py - 设置应用级图标"""
import sys, os
from PyQt5.QtWidgets import QApplication
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import Qt

# 资源路径（适配 .app 打包后的 Resources 目录）
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
ICON_PATH = os.path.join(BASE_DIR, "assets", "pet_gif", "tuanzi_icon.png")

```

```

# 或者使用 icns 文件 (macOS 原生效果更好)
# ICON_PATH = os.path.join(BASE_DIR, "assets", "tuanzi.icns")

def main():
    # 高 DPI 支持
    QApplication.setAttribute(Qt.AA_EnableHighDpiScaling, True)
    QApplication.setAttribute(Qt.AA_UseHighDpiPixmaps, True)

    app = QApplication(sys.argv)
    app.setApplicationName("桌面宠物")

    # ===== 关键: 设置应用图标 =====
    if os.path.exists(ICON_PATH):
        app.setWindowIcon(QIcon(ICON_PATH))
        print(f"    应用图标已加载: {ICON_PATH}")
    else:
        print(f"    图标文件不存在: {ICON_PATH}")

    from pet_window import PetWindow
    pet = PetWindow()
    pet.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()

```

单独窗口设置图标 (非全局继承)

```

"""pet_window.py - 为宠物主窗口单独设置图标"""

from PyQt5.QtGui import QIcon

```

```
import os, config

class PetWindow(QWidget):
    def __init__(self):
        super().__init__()
        self._setup_base()
        # ...

        # 设置窗口图标 (Dock 与 Mission Control 中显示)
        icon_path = os.path.join(config.ASSETS_DIR, "pet_gif", "tuanzi_icon.png")
        if os.path.exists(icon_path):
            self.setWindowIcon(QIcon(icon_path))
```

5.2 对话框/弹窗图标

glass_widgets.py 中的毛玻璃弹窗使用 emoji 作为功能图标，无需图片：

```
"""glass_widgets.py - 功能面板打开时显示对应 emoji 图标"""

TOOL_PANEL_BUBBLES = {
    "todo": "一起看看今天要做些什么吧 ~ 📝",
    "calc": "需要算什么？我来帮你 ~ 🧮",
    "notes": "随手记下重要的事情吧 ~ 📖",
    "shortcuts": "常用快捷键都在这里啦 ~ ⌨️",
    "reminder_settings": "来调整喝水和久坐提醒吧 ~ ⌚",
}

# 对应 Vue 前端 GlassModal 中的图标
# <GlassModal v-model="show" title="待办清单" icon="📝" :width="420">
```

标准 QMessageBox 图标用法示例：

```
"""视频素材缺失弹窗示例"""

from PyQt5.QtWidgets import QMessageBox
from PyQt5.QtGui import QIcon
```

```

import config, os

icon_path = os.path.join(config.ASSETS_DIR, "pet_gif", "tuanzi_icon.png")
msg = QMessageBox()
msg.setWindowTitle("视频素材缺失")

# 方法 1: 使用自定义图标
if os.path.exists(icon_path):
    msg.setWindowIcon(QIcon(icon_path))
    msg.setIconPixmap(QIcon(icon_path).pixmap(64, 64))

# 方法 2: 使用系统内置图标（无需资源文件）
# msg.setIcon(QMessageBox.Warning) # 黄色感叹号
# msg.setIcon(QMessageBox.Critical) # 红色错误
# msg.setIcon(QMessageBox.Information) # 蓝色信息

msg.setText("以下动作缺少视频文件...")
msg.exec_()

```

5.3 菜单项图标与 Emoji

pet_window.py 右键菜单使用 Emoji Unicode 作为图标（跨平台兼容，无需额外资源）：

```

"""pet_window.py - 右键菜单配置（Emoji 图标）"""
def _create_context_menu(self):
    menu = QMenu(self)

    # 14 项功能菜单，每项前缀 Emoji 作为图标
    menu.addAction("  AI 对话", self._open_ai_chat)
    menu.addAction("  GIF 素材测试面板", self._open_gif_test)
    menu.addSeparator()

```

```

menu.addAction("  手动结束专注", self._focus_end).setData("danger")

menu.addAction("  轻度专注",    lambda: self._set_focus("light"))

menu.addAction("  深度专注",    lambda: self._set_focus("deep"))

menu.addSeparator()

menu.addAction("  待办清单",    lambda: self._open_glass_dialog("todo"))

menu.addAction("  计算器",      lambda: self._open_glass_dialog("calc"))

menu.addAction("  随笔记事本",  lambda: self._open_glass_dialog("notes"))

menu.addAction("  Mac 快捷键面板", lambda: self._open_glass_dialog("shortcuts"))

menu.addAction("  健康提醒设置", lambda:

self._open_glass_dialog("reminder_settings"))

menu.addAction("  喝水打卡",    self._log_water)

menu.addAction("  修改宠物昵称", self._set_pet_name)

menu.addAction("  交互说明",    self._show_guide)

menu.addSeparator()

exit_action = menu.addAction("  退出", self.close)


return menu

```

【提示】Emoji 图标的优势：零资源体积、跨平台原生渲染、无需适配 Retina。若需品牌化像素图标，可改用 QAction.setIcon(QIcon("xxx.png"))。

使用 QIcon + PNG 的品牌化菜单

```

"""进阶：使用 PNG 图标替代 Emoji"""

from PyQt5.QtGui import QIcon


def _create_context_menu_branded(self):

    menu = QMenu(self)

    assets = os.path.join(config.ASSETS_DIR, "icons")


    # 每个菜单项单独设置 PNG 图标

```

```

action = QAction(QIcon(os.path.join(assets, "ai.png")), "AI 对话", self)
action.triggered.connect(self._open_ai_chat)

menu.addAction(action)

action = QAction(QIcon(os.path.join(assets, "todo.png")), "待办清单", self)
action.triggered.connect(lambda: self._open_glass_dialog("todo"))

menu.addAction(action)

# ... 同理设置其他 12 项

```

5.4 前端网页图标配置

Vue 前端作品集网页使用 `tuanzi_icon.png` 作为各组件统一图标:

favicon 浏览器标签页图标

```

<!-- index.html -->

<head>

  <link rel="icon" href="/assets/pet_gif/tuanzi_icon.png" />

  <title>团子陪伴 · 3D 毛绒猫咪桌面萌宠</title>

</head>

```

各 Vue 组件中引用

```

// NavBar.vue - 顶部导航栏 Logo

const iconUrl = import.meta.env.BASE_URL + 'assets/pet_gif/tuanzi_icon.png'
// 模板中使用: 

// ContextMenu.vue - 右键菜单头部图标

const iconUrl = import.meta.env.BASE_URL + 'assets/pet_gif/tuanzi_icon.png'
// 模板中使用: 

// AiChatPanel.vue - AI 对话面板头像

const iconUrl = import.meta.env.BASE_URL + 'assets/pet_gif/tuanzi_icon.png'
// 模板中使用: 

// GuideModal.vue / Portfolio.vue 同理

```

六、Info.plist 图标配置

6.1 CFBundleIconFile 配置

在 Info.plist 中指定 .app 包的图标文件名（放在 Contents/Resources 下）：

```
<!-- build_app.sh 中修改 Info.plist 生成部分 -->
cat > "${APP_DIR}/Contents/Info.plist" << PLIST
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <!-- 基础元数据 -->
    <key>CFBundleName</key>
    <string>桌面宠物</string>
    <key>CFBundleDisplayName</key>
    <string>桌面宠物</string>
    <key>CFBundleIdentifier</key>
    <string>com.desktoppet.app</string>
    <key>CFBundleVersion</key>
    <string>7.0.0</string>
    <key>CFBundleShortVersionString</key>
    <string>7.0.0</string>

    <!-- ===== 关键：应用图标 ===== -->
    <!-- 文件名放在 Contents/Resources/ 下，不需要写 .icns 后缀 -->
    <key>CFBundleIconFile</key>
    <string>tuanzi</string>

    <key>CFBundleExecutable</key>
    <string>桌面宠物</string>
    <key>CFBundlePackageType</key>
```

```
<string>APPL</string>
<key>NSHighResolutionCapable</key>
<true/>
<!-- 不显示 Dock 图标（桌宠无主窗口） -->
<key>LSUIElement</key>
<true/>
</dict>
</plist>
PLIST
```

【注意】CFBundleIconFile 的值 tuanzi 对应 Contents/Resources/tuanzi.icns。只写文件名，不要写 .icns 后缀!

6.2 完整配置示例 + icns 复制到 .app

```
# 在 build_app.sh 中添加 icns 复制步骤
# 位置：复制项目文件之后（第 5 步和第 6 步之间）

# ===== 新增：复制 icns 图标到 Resources =====
ICNS_SRC="assets/tuanzi.icns"
if [ -f "${ICNS_SRC}" ]; then
    cp "${ICNS_SRC}" "${APP_DIR}/Contents/Resources/tuanzi.icns"
    echo "  已复制应用图标: tuanzi.icns"
else
    echo "  未找到图标源文件: ${ICNS_SRC}"
    echo "  请将 tuanzi.icns 放入 assets/ 目录，或跳过此步（使用默认系统图标）"
fi

# 完整验证 .app 中的图标结构
find dist/桌面宠物.app -name "*.icns" -o -name "Info.plist" | sort
# 预期输出：
# dist/桌面宠物.app/Contents/Info.plist
# dist/桌面宠物.app/Contents/Resources/tuanzi.icns
```



```
# 验证 Info.plist 中 CFBundleIconFile 是否正确
/usr/libexec/PlistBuddy -c "Print :CFBundleIconFile" dist/桌面宠物.app/Contents/Info.plist
# 预期输出: tuanzi
```

七、PyInstaller 打包命令详解

7.1 基础打包命令

0. 安装 PyInstaller

```
pip install pyinstaller
```

```
# ===== 方式 A: 单目录模式 (推荐调试用) =====
```

```
# --windowed 不显示命令行黑框 (GUI 应用必备)
```

```
# --name 生成的 .app 名称
```

```
# --icon 应用图标 (.icns)
```

```
# --add-data 打包资源文件 (格式: 源路径:目标路径, macOS 用 : 分隔)
```

```
pyinstaller --windowed --name "桌面宠物" --icon "assets/tuanzi.icns" --add-data
"assets:assets" --add-data "animations.json:" --add-data "config.py:" --add-data
"pet_window.py:" --add-data "ai_chat.py:" --add-data "reminder.py:" --add-data
"tools_panel.py:" --add-data "glass_widgets.py:" main.py
```

```
# ===== 方式 B: 单文件模式 (分发时体积更小) =====
```

```
# --onefile 所有依赖打包到单个可执行文件
```

```
# 注意: 单文件模式启动慢 (需解压到临时目录), 资源文件用 sys._MEIPASS 访问
```

```
pyinstaller --onefile --windowed --name "桌面宠物" --icon "assets/tuanzi.icns" --
add-data "assets:assets" --add-data "animations.json:" --add-data "config.py:" --add-
data "pet_window.py:" --add-data "ai_chat.py:" --add-data "reminder.py:" --add-data
"tools_panel.py:" --add-data "glass_widgets.py:" main.py
```

【注意】PyInstaller 打包后, 代码中访问资源需使用 `sys._MEIPASS` (单文件临时目录) 或 `os.path.dirname(sys.executable)` (单目录模式)。本项目 `build_app.sh` 采用 `venv` 自举方案, 不依赖 PyInstaller, 故无需修改资源路径。

main.py 适配 PyInstaller 的资源路径写法

```
"""main.py - PyInstaller 兼容资源路径模板 (供参考)"""
import sys, os
```

```

def resource_path(relative_path):
    """获取资源文件路径，兼容 PyInstaller --onefile / --onedir / 开发模式"""
    try:
        # PyInstaller 临时解压目录（单文件模式）
        base_path = sys._MEIPASS
    except AttributeError:
        try:
            # PyInstaller 单目录模式 / macOS .app
            base_path = os.path.dirname(sys.executable)
            # .app 中 sys.executable = Contents/MacOS/app
            # 资源在 Contents/Resources
            if "Contents/MacOS" in base_path:
                base_path = os.path.join(
                    os.path.dirname(base_path), "Resources"
                )
        except AttributeError:
            # 开发模式
            base_path = os.path.dirname(os.path.abspath(__file__))
    return os.path.join(base_path, relative_path)

# 使用
ASSETS_DIR = resource_path("assets")
CONFIG_PATH = resource_path("config.py")

```

7.2 打包参数速查表

参数	说明	本项目建议值
-F / --onefile	打包为单个可执行文件	否（启动慢）
-D / --onedir	打包为目录（默认）	推荐
-w / --windowed / --	无命令行黑框	必须

noconsole		
-n / --name NAME	输出文件名	桌面宠物
-i / --icon ICON	应用图标	assets/tuanzi.icns
--add-data SRC:DST	打包资源文件	assets:assets 等
--hidden-import MOD	强制导入模块	PyQt5.sip
--collect-submodules PKG	收集子模块	PyQt5
--collect-data PKG	收集包数据	PyQt5
--noconfirm	覆盖 build/dist 不询问	打包脚本用
--clean	清理缓存后构建	打包脚本用
--log-level LEVEL	日志级别	INFO (调试用 DEBUG)
--osx-bundle-identifier ID	macOS Bundle ID	com.desktoppet.app
--osx-emulate-shell-env	模拟 shell 环境变量	macOS 打包时

7.3 Spec 文件配置

首次运行 PyInstaller 会生成 桌面宠物.spec，之后修改 spec 可重复构建：

```
# desktop_pet.spec - PyInstaller 配置文件
# -*- mode: python ; coding: utf-8 -*-
```

```
block_cipher = None
```

```
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    # 打包资源
    datas=[
        ('assets', 'assets'),
        ('animations.json', '.'),
        ('config.py', '.'),
        ('pet_window.py', '.'),
```

```
( 'ai_chat.py', '.'),
( 'reminder.py', '.'),
( 'tools_panel.py', '.'),
( 'glass_widgets.py', '.'),
( 'requirements.txt', '.'),
],
# 隐藏导入
hiddenimports=[
    'PyQt5.sip',
    'PyQt5.QtMultimedia',
    'PyQt5.QtOpenGL',
],
hookspath=[],
hooksconfig={},
runtime_hooks=[],
# 排除不需要的模块（减小体积）
excludes=[
    'tkinter', 'matplotlib', 'pandas', 'numpy',
    'scipy', 'pytest', 'unittest',
],
win_no_prefer_redirects=False,
win_private_assemblies=False,
cipher=block_cipher,
noarchive=False,
)

pyz = PYZ(a.pure, a.zipped_data, cipher=block_cipher)

exe = EXE(
    pyz,
```

```
a.scripts,  
[],  
exclude_binaries=True,  
name='桌面宠物',  
debug=False,  
bootloader_ignore_signals=False,  
strip=False,  
upx=True,      # 使用 UPX 压缩 (需 brew install upx)  
console=False, # 必须 False, GUI 无黑框  
disable_windowed_traceback=False,  
target_arch=None,  
codesign_identity='-', # ad-hoc 签名  
entitlements_file=None,  
icon='assets/tuanzi.icns',  
)
```

```
coll = COLLECT(  
    exe,  
    a.binaries,  
    a.zipfiles,  
    a.datas,  
    strip=False,  
    upx=True,  
    upx_exclude=[],  
    name='桌面宠物',  
)
```

```
# macOS .app bundle
```

```
app = BUNDLE(  
    coll,
```

```

name='桌面宠物.app',
icon='assets/tuanzi.icns',
bundle_identifier='com.desktoppet.app',
info_plist={
    'CFBundleShortVersionString': '7.0.0',
    'CFBundleVersion': '7.0.0',
    'NSHighResolutionCapable': True,
    'LSUIElement': True,
    'NSHumanReadableCopyright': '© 2026 团子陪伴',
},
)

# 使用 spec 文件构建（比每次写一长串参数方便）
pyinstaller --noconfirm --clean desktop_pet.spec

```

输出位置

dist/桌面宠物/ （单目录）

dist/桌面宠物.app （macOS Bundle）

八、py2app 打包（macOS 原生）

8.1 setup.py 配置

py2app 是 Python 官方推荐的 macOS 打包工具，对 Cocoa API 兼容性更好：

```

"""setup.py - py2app 配置文件"""

from setuptools import setup

APP = ['main.py']

DATA_FILES = [

    # 资源文件原样复制到 Contents/Resources
    ('', ['animations.json', 'requirements.txt', 'config.py']),
    ('assets', [
        'assets/pet_gif/idle.gif',
        'assets/pet_gif/happy.gif',

```

```
'assets/pet_gif/lie_back.gif',
'assets/pet_gif/rest.gif',
'assets/pet_gif/stretch.gif',
'assets/pet_gif/wave.gif',
'assets/pet_gif/focus.gif',
'assets/pet_gif/tuanzi_icon.png',
]),
('assets/video', [
    'assets/video/eat.mp4',
    'assets/video/stretch.mp4',
    'assets/video/lie_back.mp4',
    'assets/video/idle.gif',
    'assets/video/happy.gif',
    'assets/video/lie_side.gif',
    'assets/video/sleep.gif',
]),
]

OPTIONS = {
    'argv_emulation': True,
    'iconfile': 'assets/tuanzi.icns', # 应用图标
    'plist': {
        'CFBundleName': '桌面宠物',
        'CFBundleDisplayName': '桌面宠物',
        'CFBundleIdentifier': 'com.desktoppet.app',
        'CFBundleVersion': '7.0.0',
        'CFBundleShortVersionString': '7.0.0',
        'NSHighResolutionCapable': True,
        'LSUIElement': True, # 不显示 Dock
        'NSHumanReadableCopyright': '© 2026 团子陪伴',
```

```

},
# 半独立模式: 不复制 Python 解释器, 依赖用户系统 Python
# (减小 ~50MB 体积, 但要求目标机器装了 Python 3.8+)
'emulate_shell_environment': True,
# 预导入 PyQt5 相关模块
'includes': [
    'PyQt5',
    'PyQt5.QtCore',
    'PyQt5.QtGui',
    'PyQt5.QtWidgets',
    'PyQt5.QtMultimedia',
    'PyQt5.QtOpenGL',
    'PyQt5.sip',
],
# 排除不必要的包
'excludes': [
    'tkinter', 'test', 'unittest', 'pytest',
    'matplotlib', 'scipy', 'pandas', 'numpy',
],
# 是否压缩 (启用后 .app 更小, 启动稍慢)
'optimize': 2,
'compressed': True,
}

setup(
    app=APP,
    name='桌面宠物',
    version='7.0.0',
    data_files=DATA_FILES,
    options={'py2app': OPTIONS},

```



```
setup_requires=['py2app'],
install_requires=['PyQt5>=5.15.0', 'requests>=2.28.0'],
)
```

8.2 py2app 打包命令

0. 安装 py2app

```
pip install py2app
```

1. Alias 模式（开发调试，秒级构建，不打包依赖）

```
python setup.py py2app -A
```

输出：dist/桌面宠物.app（别名模式，资源引用源文件，方便调试但不能分发）

2. 生产模式（完整打包）

```
python setup.py py2app
```

输出：dist/桌面宠物.app（完整自包含，体积约 150~250MB）

3. 清理旧构建

```
rm -rf build dist && python setup.py py2app
```

4. 验证

```
codesign --verify --deep dist/桌面宠物.app
```

输出：dist/桌面宠物.app: valid on disk

dist/桌面宠物.app: satisfies its Designated Requirement

九、build_app.sh 脚本打包（本项目推荐）

9.1 一键打包命令

进入项目目录

```
cd "/Users/suyue/Desktop/材料/desktop_pet 5"
```

一键打包

```
bash build_app.sh
```

```

# 预期输出（完整）：
# =====
#     桌面宠物 - macOS .app 打包
# =====
#     Python 3.11
#     清理旧构建...
#     构建 .app...
#     复制项目文件...
#     签名...
#     签名成功
#
#     打包成功!
#
#     位置: dist/桌面宠物.app
#
# 使用方式:
# 1. 双击运行
# 2. 拖到 /Applications/ 永久使用
# 3. 首次启动会自动安装依赖（约 30 秒）

```

9.2 脚本参数说明

配置项	位置（行号）	说明
APP_NAME	第 20 行	应用名称（菜单中显示）
APP_DIR	第 21 行	输出路径 dist/\${APP_NAME}.app
复制的资源文件	第 46~56 行	main.py / pet_window.py / assets / 等
Info.plist Bundle ID	第 224 行	com.desktoppet.app
Info.plist 版本号	第 226、228 行	7.0.0（构建号+显示版本）
Info.plist LSUIElement	第 236 行	true（无 Dock 图标）

签名方式	第 243 行	<code>codesign --force --deep --sign - (ad-hoc)</code>
------	---------	--

9.3 与 PyInstaller/py2app 对比

对比项	build_app.sh	PyInstaller	py2app
原理	启动脚本+venv 自举	Bootloader+二进制 打包	Cocoa Bundle+Python 嵌入
.app 体积	~5MB (不含 venv)	~180~250MB	~150~220MB
首次启动耗时	30~60 秒 (装依赖)	正常启动 (<5 秒)	正常启动 (<5 秒)
可移植性	任意移动 (动态路径)	完整自包含	完整自包含
本项目选择	推荐	备选 (需改资源路径)	备选 (需 setup.py)

十、代码签名与公证流程

10.1 Ad-hoc 签名 (无开发者账号)

```
# 对 .app 进行 ad-hoc 空签名
codesign --force --deep --sign - "dist/桌面宠物.app"

# --force 覆盖已有签名
# --deep 递归签名 .app 下所有 frameworks/dylib/可执行文件
# --sign - 空签名身份 (无需证书)

# 验证签名
codesign --verify --deep --strict "dist/桌面宠物.app"
# 成功: 无输出 (exit code 0)
# 失败: 输出具体错误信息

# 查看签名详情
codesign -dvvv "dist/桌面宠物.app"
```

10.2 Developer ID 签名 (Apple 开发者账号 \$99/年)

0. 前提:

- # - 已注册 Apple Developer Program
- # - 已在 Keychain Access 中安装 Developer ID Application 证书
- # - 证书名称形如: Developer ID Application: Your Name (TEAMID123)

1. 查看已安装的证书

```
security find-identity -v -p codesigning
```

从输出中复制 Developer ID Application 的完整名称

2. 使用 Developer ID 签名 (启用 Hardened Runtime)

```
codesign --force --deep --sign "Developer ID Application: Your Name (TEAMID123)" --options runtime --timestamp "dist/桌面宠物.app"
```

--options runtime 启用 Hardened Runtime (公证必须)

--timestamp 加时间戳 (防止证书过期导致签名失效)

3. 验证

```
codesign --verify --deep --strict "dist/桌面宠物.app"
```

```
codesign -dvvv "dist/桌面宠物.app" | grep "Developer ID"
```

10.3 Notarization 公证 (提交 Apple 扫描)

0. 准备:

- # - Apple ID (开启 2FA)
- # - App-Specific Password (appleid.apple.com 生成, 格式: xxxx-xxxx-xxxx-xxxx)
- # - Team ID (developer.apple.com/account 中查看)

1. 先打包成 DMG/ZIP (必须先签名再公证)

```
hdiutil create -volname "桌面宠物" -srcfolder "dist/桌面宠物.app" -ov -format UDZO "桌面宠物-v7.0.0.dmg"
```

2. 提交公证 (使用 Xcode 13+ 自带 notarytool)

```
xcrun notarytool submit "桌面宠物-v7.0.0.dmg" --apple-id "your@apple-id.com" --team-id "TEAMID123" --password "xxxx-xxxx-xxxx-xxxx" --wait
```

--wait: 同步等待公证完成 (否则需轮询查询)

成功输出: Processing complete

Status: Accepted

id: 01234567-89ab-cdef-0123-456789abcdef

3. 查询历史记录

```
xcrun notarytool history --apple-id "your@apple-id.com" --team-id "TEAMID123" --password "xxxx-xxxx-xxxx-xxxx"
```

4. 查看某次详细日志

```
xcrun notarytool log <submission-id> --apple-id "your@apple-id.com" --team-id "TEAMID123" --password "xxxx-xxxx-xxxx-xxxx"
```

【注意】 公证被拒的常见原因: 未启用 Hardened Runtime (`--options runtime`)、包含未签名的第三方 dylib、使用过时的 API。请查看 `notarytool log` 的详细错误报告。

10.4 Stapler 装订公证结果

公证通过后, 将 ticket 装订到 DMG/.app

装订后, 离线环境也能验证通过, 无需联网查询

装订 DMG (分发文件)

```
xcrun stapler staple "桌面宠物-v7.0.0.dmg"
```

装订 .app

```
xcrun stapler staple "dist/桌面宠物.app"
```

验证装订结果

```
xcrun stapler validate "桌面宠物-v7.0.0.dmg"
```

```
# 成功: The staple and validate action worked!

# 最终验证: Gatekeeper 全链路通过
spctl --assess --type execute -vvv "桌面宠物-v7.0.0.dmg"

# 成功输出: source=Notarized Developer ID
#      origin=Developer ID Application: Your Name (TEAMID123)
```

十一、DMG 安装包制作

11.1 hdiutil 命令制作 (系统自带)

```
# 方式一: 基本 DMG (无自定义背景/图标布局)

hdiutil create -volname "桌面宠物 v7.0.0" -srcfolder "dist/桌面宠物.app" -ov -format
UDZO "桌面宠物-v7.0.0.dmg"

# 参数说明:

# -volname DMG 挂载后的卷标名 (Finder 中显示)
# -srcfolder 要打包的源目录 (直接放入 .app 即可, 用户拖到 /Applications)
# -ov      覆盖已存在的同名文件
# -format UDZO 压缩格式 (最常用, 读性能好)
# 其他格式: UDRW 可写、UDRO 只读、UDCO 更强压缩

# 挂载测试

hdiutil attach "桌面宠物-v7.0.0.dmg"

ls /Volumes/桌面宠物\ v7.0.0/

# 预期输出: 桌面宠物.app

# 推出 (测试完记得卸载)

hdiutil detach /Volumes/桌面宠物\ v7.0.0/
```

11.2 create-dmg 工具美化 (推荐)

```
# 0. 安装 create-dmg

brew install create-dmg
```

1. 美化版 DMG 生成

```
create-dmg --volname "桌面宠物 v7.0.0" --window-pos 200 120 --window-size 600 380 --icon-size 100 --icon "桌面宠物.app" 175 190 --app-drop-link 425 190 --volicon "assets/tuanzi.icns" --background "assets/dmg_background.png" --no-internet-enable "桌面宠物-v7.0.0.dmg" "dist/桌面宠物.app"
```

参数说明:

```
# --window-pos      DMG 窗口打开时的位置
# --window-size     DMG 窗口尺寸
# --icon-size       图标大小 (像素)
# --icon "xxx.app" X Y  .app 图标的坐标
# --app-drop-link X Y  /Applications 快捷方式的坐标
# --volicon         DMG 卷标图标 (Finder 中显示)
# --background      DMG 窗口背景图 (建议 600x380 PNG)
# --no-internet-enable 禁用互联网启用 (与公证不兼容时加)
```

十二、常见问题排查

12.1 图标不显示

Finder 中 .app 图标是默认白纸:

- 检查 CFBundleIconFile 是否正确配置 (不加 .icns 后缀)
- 确认 tuanzi.icns 是否真的在 Contents/Resources/ 下
- 执行 touch dist/桌面宠物.app 强制刷新 Finder 缓存
- 按 Option 右键 Finder → 「重新开启」后再查看

Dock 中不显示图标 (空白或问号) :

- LSUIElement=true 的应用本就不显示 Dock 图标 (桌宠设计如此)
- 若需显示 Dock 图标, 将 Info.plist 中 LSUIElement 设为 false
- 检查 main.py 中是否调用了 app.setWindowIcon(QIcon(...))

窗口标题栏图标缺失:

- 检查 PNG 路径是否正确 (用 os.path.exists() 打印确认)
- 确认 PNG 不是损坏文件, 用预览.app 打开测试

- 换 512x512 以上尺寸的 PNG 试试 (Retina 屏可能放大后模糊)

12.2 打包启动失败

错误: No module named config / pet_window

【提示】 启动脚本中 cd 到 Resources 后再执行 python main.py, 确保同目录模块能被 import。build_app.sh 已正确设置, 请检查启动脚本的 cd "\${RES_DIR}" 行。

错误: Could not find the Qt platform plugin "cocoa"

1. 检查 Qt 插件路径是否导出

```
grep "QT_PLUGIN_PATH" ~/Library/Application\ Support/桌面宠物/launch.log
```

2. 若为空, 手动在 venv 中测试

```
~/venv/bin/python -c "  
from PyQt5.QtCore import QLibraryInfo  
print(QLibraryInfo.location(QLibraryInfo.PluginsPath))  
"
```

3. 重装 PyQt5 (强制不缓存)

```
pip uninstall -y PyQt5 PyQt5-Qt5 PyQt5-sip  
pip install --no-cache-dir PyQt5
```

双击 .app 一闪而过 (命令行运行有报错)

直接从终端运行可执行文件, 看到完整报错

```
dist/桌面宠物.app/Contents/MacOS/桌面宠物
```

或者看日志

```
tail -100 ~/Library/Application\ Support/桌面宠物/launch.log
```

12.3 签名与公证错误

错误信息	解决方案
code object is not signed at all	加 --deep 参数递归签名: codesign --force --deep --sign - app
resource envelope is obsolete	升级 codesign, 或使用 --strict 重新签名

The signature is invalid	修改 .app 内文件后必须重新签名；不要往已签名的 .app 里增删文件
Notarization rejected: Unsafe	检查 notarytool log，通常是缺少 Hardened Runtime，加 --options runtime 重签后再提交

十三、附录：完整打包命令速查

操作	命令
安装 Pillow (图标处理)	<code>pip install pillow</code>
批量生成 .iconset 尺寸	<code>sips -z 16 16 master.png --out icon.iconset/icon_16x16.png</code> (全套 10 条)
生成 icns	<code>iconutil -c icns icon.iconset -o tuanzi.icns</code>
生成 Windows ICO	<code>Pillow: images[0].save("tuanzi.ico", sizes=sizes, append_images=...)</code>
开发模式运行	<code>bash run.sh</code> 或 <code>python main.py</code>
build_app.sh 打包	<code>bash build_app.sh</code>
查看启动日志	<code>tail -f ~/Library/Application\ Support/桌面宠物/launch.log</code>
安装 PyInstaller	<code>pip install pyinstaller</code>
PyInstaller 单目录打包	<code>pyinstaller --windowed --name 桌面宠物 --icon assets/tuanzi.icns --add-data "assets:assets" main.py</code>
用 Spec 构建	<code>pyinstaller --noconfirm --clean desktop_pet.spec</code>
安装 py2app	<code>pip install py2app</code>
py2app 别名模式 (调试)	<code>python setup.py py2app -A</code>
py2app 生产模式	<code>python setup.py py2app</code>
Ad-hoc 签名	<code>codesign --force --deep --sign - dist/桌面宠物.app</code>
Developer ID 签名	<code>codesign --force --deep --sign "Developer ID ..." --options runtime --timestamp app</code>
验证签名	<code>codesign --verify --deep --strict dist/桌面宠物.app</code>
制作基础 DMG	<code>hdiutil create -volname "桌面宠物" -srcfolder dist/桌面宠物.app -format UDZO v7.dmg</code>
提交公证	<code>xcrun notarytool submit v7.dmg --apple-id xxx --team-id</code>

	xxx --password xxx --wait
装订公证结果	xcrun stapler staple v7.dmg
Gatekeeper 全链路验证	spctl --assess --type execute -vvv v7.dmg